

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА» (БГТУ им. В.Г. Шухова)
Кафедра программного обеспечения
вычислительной техники и автоматизированных
систем

Лабораторная работа №4
по дисциплине: Архитектура вычислительных систем
тема: «**Команды передачи управления**»

Выполнил: студент группы ВТ-231
Масленников Д. А.
Проверили:
Осипов О.В.
Бондаренко Т.В.

Белгород 2025

Цель работы: изучение команд перехода для организации циклов и ветвлений, получение навыков создания процедур с аргументами.

Вариант 14

Задания для выполнения к работе

1. Написать программу для вычисления значения арифметического выражения, используя команды условного и безусловного перехода согласно варианту задания. Подобрать набор тестовых данных (не менее 3). При выполнении операций с числами, преобразовывать их к 4-байтовым числам со знаком. Результат вывести на экран.
2. Написать программу для вычисления значения арифметического выражения, содержащего функцию. Вычисление функции организовать в виде отдельной подпрограммы по всем правилам, описанным выше. Для обработки массивов использовать команды для работы с циклами и команды условного перехода. Подобрать набор тестовых данных (не менее 3). Результат вывести на экран.
3. Написать программу для решения задачи из документа **task3.pdf**. Номер задачи выбрать по формуле: $k \bmod 8$, где k – номер варианта по журналу.

14	$p = \begin{cases} x^2 + 1 + z, & x > 3, \\ \frac{x}{(4 - z^2)}, & x \in [-3; 3] \text{ и } z \notin [-2; 2], \\ x - y - 2z, & x \in [-3; 3] \text{ и } z \in [-2; 2] \end{cases}$	x – знаковое однобайтовое y – знаковое 2-байтовое z – беззнаковое 2-байтовое
----	--	--

14	$r = \sum_{i=0}^n x_i + \sum_{i=0}^n \begin{cases} \frac{x_i}{y_i} + f(x_i, y_i), & i - \text{чётно}, \\ x_i + y_i^2 + f(x_i^2, y_i^2), & i - \text{нечётно}, \end{cases}$ $f(x, y) = \begin{cases} \frac{ x + y }{200}, & x > 7, \\ (x - y)^3, & x \leq 7 \end{cases}$	x – массив 4-байтовых знаковых чисел y – массив 2-байтовых беззнаковых чисел n – беззнаковая переменная размером 2 байта
----	---	---

Задание 1:

section .data

x db -3 ; знаковое однобайтовое
y dw -5 ; знаковое 2-байтовое
z dw 10 ; беззнаковое 2-байтовое

format db "p = %d", 10, 0

section .text

global _start
extern printf
extern exit

_start:

; Проверка условия $|x| > 3$
XOR EAX, EAX ; Обнулить EAX
MOV AL, [x] ; AL = x
CBW ; Расширить до 2 байт (AX)
CWDE ; Расширить до 4 байт (EAX)

; Вычисляем $|x|$
MOV EBX, EAX ; Сохраняем x в EBX
CMP EAX, 0
JGE abs_done ; Если $x \geq 0$, то $|x| = x$
NEG EAX ; Иначе $|x| = -x$

abs_done:

; Проверяем $|x| > 3$
CMP EAX, 3
JG case1 ; Если $|x| > 3$, переход к case1

; Иначе $x \in [-3, 3]$
; Нужно проверить z
XOR ECX, ECX
MOV CX, [z] ; ECX = z (беззнаковое, просто загружаем)

; Проверяем $z \in [-2, 2]$ для беззнакового z это $z \in [0, 2]$
CMP ECX, 2
JBE case3 ; Если $z \leq 2$, то $z \in [0, 2] \rightarrow$ case3

; Иначе $z > 2$, т.е. $z \notin [-2, 2] \rightarrow$ case2
JMP case2

case1:

; $p = x^2 + 1 + z$

```

MOV EAX, EBX      ; EAX = x
IMUL EAX          ; EDX:EAX = x^2
ADD EAX, 1        ; EAX = x^2 + 1
XOR ECX, ECX
MOV CX, [z]       ; ECX = z
ADD EAX, ECX      ; EAX = x^2 + 1 + z
JMP output

```

case2:

```

; p = x / (4 - z^2)
XOR ECX, ECX
MOV CX, [z]      ; ECX = z
MOV EAX, ECX     ; EAX = z
IMUL EAX         ; EDX:EAX = z^2
MOV ECX, EAX     ; ECX = z^2
MOV EAX, 4       ; EAX = 4
SUB EAX, ECX     ; EAX = 4 - z^2
MOV ECX, EAX     ; ECX = делитель (4 - z^2)

MOV EAX, EBX     ; EAX = x
CDQ              ; Расширить EAX до EDX:EAX для деления
IDIV ECX         ; EAX = x / (4 - z^2)
JMP output

```

case3:

```

; p = x - y - 2z
MOV EAX, EBX     ; EAX = x
XOR ECX, ECX
MOV CX, [y]      ; ECX = y
CWDE            ; Расширить CX до ECX со знаком
MOVSX ECX, word [y] ; ECX = y (знаковое расширение)
SUB EAX, ECX     ; EAX = x - y

XOR ECX, ECX
MOV CX, [z]      ; ECX = z
SHL ECX, 1       ; ECX = 2*z
SUB EAX, ECX     ; EAX = x - y - 2z

```

output:

```

; Вывод результата
push eax
push format
call printf
add esp, 8

```

; Выход из программы
push 0
call exit

Тестовые данные:

x	y	z	условие	Формула	p
-5	10	5	$ x > 3$	$P = xp = x^2 + 1 + z$	$p = 25 + 1 + 5 = \mathbf{31}$
2	8	1	$x \in [-3,3]$ и $z \in [0,2]$	$p = x - y - 2z$	$p = 2 - 8 - 2 = \mathbf{-8}$
-3	-5	10	$x \in [-3,3]$ и $z \notin [0,2]$	$p = x / (4 - z^2)$	$p = -3 / (4 - 100) = -3 / (-96) = \mathbf{0}$

Результат выполнения программы для разных тестовых данных:

1 Тест

```
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > nasm -f elf32 lab4_task1.asm -o lab4_task1.o
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ld -m elf_i386 -dynamic-linker /lib/ld-linux.so.2 \
-o lab4_task1 lab4_task1.o \
-lc -L/usr/lib32
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ./lab4_task1
p = 31
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > █
```

2 Тест

```
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > nasm -f elf32 lab4_task1.asm -o lab4_task1.o
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ld -m elf_i386 -dynamic-linker /lib/ld-linux.so.2 \
-o lab4_task1 lab4_task1.o \
-lc -L/usr/lib32
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ./lab4_task1
p = -8
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > █
```

3 Тест

```
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > nasm -f elf32 lab4_task1.asm -o lab4_task1.o
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ld -m elf_i386 -dynamic-linker /lib/ld-linux.so.2 \
-o lab4_task1 lab4_task1.o \
-lc -L/usr/lib32
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ./lab4_task1
p = 0
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > █
```

Задание 2:

section .data

x dd 10, -5 ; массив 4-байтовых знаковых чисел
y dw 2, 10 ; массив 2-байтовых беззнаковых чисел
n dw 1 ; количество элементов (от 0 до n, т.е. n+1 элементов)

format db "r = %d", 10, 0

section .bss

sum1 resd 1 ; Первая сумма
sum2 resd 1 ; Вторая сумма

section .text

global main
extern printf
extern exit

; Функция f(x, y)

; Аргументы: [ebp+8] = x (4 байта), [ebp+12] = y (4 байта)

; Возвращает результат в EAX

f:

push ebp
mov ebp, esp
push ebx

mov eax, [ebp+8] ; EAX = x
mov ebx, [ebp+12] ; EBX = y

; Проверяем $x > 7$

cmp eax, 7

jg f_case1 ; Если $x > 7$

; Иначе $x \leq 7$: $f(x,y) = (x - y)^3$

sub eax, ebx ; EAX = $x - y$

mov ebx, eax ; EBX = $x - y$

imul eax, ebx ; EAX = $(x-y)^2$

imul eax, ebx ; EAX = $(x-y)^3$

jmp f_end

f_case1:

; $f(x,y) = |x + y| / 200$

add eax, ebx ; EAX = $x + y$

; Вычисляем $|x + y|$

cmp eax, 0

```
jge f_abs_done
neg eax          ; Если отрицательное, делаем положительным
f_abs_done:
```

```
; Делим на 200
cdq              ; Расширяем EAX до EDX:EAX
mov ebx, 200
idiv ebx         ; EAX = |x + y| / 200
```

```
f_end:
pop ebx
pop ebp
ret 8            ; Очищаем стек от 8 байт
```

```
main:
push ebp
mov ebp, esp

; Обнуляем суммы
mov dword [sum1], 0
mov dword [sum2], 0

xor esi, esi     ; ESI = индекс i
xor ecx, ecx
mov cx, [n]      ; ECX = n
inc ecx          ; ECX = n + 1 (количество итераций)
```

```
loop_start:
; Первая сумма: накапливаем x[i]
mov eax, [x + esi*4] ; EAX = x[i]
add [sum1], eax      ; sum1 += x[i]

; Проверяем четность i
test esi, 1          ; Проверяем младший бит
jnz odd_case         ; Если i нечётное
```

```
even_case:
; i чётное: x_i / y_i + f(x_i, y_i)
mov eax, [x + esi*4] ; EAX = x[i]
xor edx, edx
movzx edx, word [y + esi*2] ; EDX = y[i]

; Сохраняем для вызова функции
push edx           ; Сохраняем y[i]
push eax           ; Сохраняем x[i]
```

```

; Деление x[i] / y[i]
cdq          ; Расширяем EAX до EDX:EAX
idiv dword [esp+4] ; Делим на y[i]
push eax     ; Сохраняем результат деления

; Вызов функции f(x[i], y[i])
; Аргументы уже в стеке (после push eax)
push dword [esp+8] ; y[i]
push dword [esp+8] ; x[i]
call f           ; Результат в EAX

pop ebx        ; EBX = результат деления (x[i]/y[i])
add esp, 8     ; Очищаем x[i] и y[i]

add eax, ebx   ; EAX = x[i]/y[i] + f(x[i], y[i])
add [sum2], eax ; sum2 += результат

jmp loop_continue

```

odd_case:

```

; i нечётное: x_i + y_i^2 + f(x_i^2, y_i^2)
mov eax, [x + esi*4] ; EAX = x[i]
add [sum2], eax     ; sum2 += x[i]

xor edx, edx
movzx edx, word [y + esi*2] ; EDX = y[i]
mov eax, edx             ; EAX = y[i]
imul eax, edx            ; EAX = y[i]^2
add [sum2], eax         ; sum2 += y[i]^2
push eax                ; Сохраняем y[i]^2

; Вычисляем x[i]^2
mov eax, [x + esi*4] ; EAX = x[i]
imul eax             ; EDX:EAX = x[i]^2

; Вызов f(x[i]^2, y[i]^2)
push dword [esp]     ; y[i]^2
push eax             ; x[i]^2
call f              ; Результат в EAX

add esp, 4          ; Убираем y[i]^2
add [sum2], eax     ; sum2 += f(x[i]^2, y[i]^2)

```

loop_continue:


```
inc esi      ; i++
dec ecx      ; ECX--
jnz loop_start ; Переход если ECX != 0
```

```
; r = sum1 + sum2
mov eax, [sum1]
add eax, [sum2]
```

```
; Вывод результата
push eax
push format
call printf
add esp, 8
```

```
; Выход из программы
push 0
call exit
```

Тестовые данные:

X	Y	N	Ожидаемый r
10, -5, 3, 8	2, 10, 5, 3	3	125
10	2	0	15
10, -5	2, 10	1	105

Вычисления тест 1:

- Первая сумма: $10 + (-5) + 3 + 8 = 16$
- $i=0$ (чётное): $10/2 + f(10,2) = 5 + 0 = 5$
- $i=1$ (нечётное): $-5 + 100 + f(25,100) = 95 + 0 = 95$
- $i=2$ (чётное): $3/5 + f(3,5)$
 - $3/5 = 0$ (целочисленное деление)
 - $f(3,5)$: $x=3 \leq 7$, $f = (3-5)^3 = (-2)^3 = -8$
 - Итого: $0 + (-8) = -8$
- $i=3$ (нечётное): $8 + 9 + f(64,9)$
 - $8 + 9 = 17$
 - $f(64,9)$: $x=64 > 7$, $f = |64+9|/200 = 73/200 = 0$
 - Итого: $17 + 0 = 17$
- Вторая сумма: $5 + 95 + (-8) + 17 = 109$
- **$r = 16 + 109 = 125$**

Вычисления тест 2:

- Первая сумма: $\sum x[i] = 10$
- $i=0$ (чётное): $x[0]/y[0] + f(10, 2)$
 - $x[0]/y[0] = 10/2 = 5$
 - $f(10, 2)$: $x=10 > 7$, поэтому $f = |10+2|/200 = 12/200 = 0$
 - Итого: $5 + 0 = 5$
- Вторая сумма: 5
- **$r = 10 + 5 = 15$**

Вычисления тест 4:

- Первая сумма: $10 + (-5) = 5$
- $i=0$ (чётное): $10/2 + f(10,2) = 5 + 0 = 5$
- $i=1$ (нечётное): $-5 + 10^2 + f(25, 100)$
 - $-5 + 100 = 95$
 - $f(25, 100)$: $x=25 > 7$, $f = |25+100|/200 = 125/200 = 0$
 - Итого: $95 + 0 = 95$
- Вторая сумма: $5 + 95 = 100$
- **$r = 5 + 100 = 105$**

Результат выполнения тестов:

```
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > gcc -m32 lab4_task2.o -o lab4_task2 -no-pie
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ./lab4_task2
r = 125
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > nasm -f elf32 lab4_task2.asm -o lab4_task2.o
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > gcc -m32 lab4_task2.o -o lab4_task2 -no-pie
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ./lab4_task2
r = 15
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > nasm -f elf32 lab4_task2.asm -o lab4_task2.o
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > gcc -m32 lab4_task2.o -o lab4_task2 -no-pie
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > ./lab4_task2
r = 105
```

Задание 3 (через наибольший общий делитель):

section .data

x dd 10, -5 ; массив 4-байтовых знаковых чисел
y dw 2, 10 ; массив 2-байтовых беззнаковых чисел
n dw 1 ; количество элементов (от 0 до n, т.е. n+1 элементов)

format db "r = %d", 10, 0

section .bss

sum1 resd 1 ; Первая сумма
sum2 resd 1 ; Вторая сумма

section .text

global main
extern printf
extern exit

; Функция f(x, y)

; Аргументы: [ebp+8] = x (4 байта), [ebp+12] = y (4 байта)

; Возвращает результат в EAX

f:

push ebp
mov ebp, esp
push ebx

mov eax, [ebp+8] ; EAX = x
mov ebx, [ebp+12] ; EBX = y

; Проверяем $x > 7$

cmp eax, 7

jg f_case1 ; Если $x > 7$

; Иначе $x \leq 7$: $f(x,y) = (x - y)^3$

sub eax, ebx ; EAX = $x - y$

mov ebx, eax ; EBX = $x - y$

imul eax, ebx ; EAX = $(x-y)^2$

imul eax, ebx ; EAX = $(x-y)^3$

jmp f_end

f_case1:

; $f(x,y) = |x + y| / 200$

add eax, ebx ; EAX = $x + y$

; Вычисляем $|x + y|$

```
    cmp eax, 0
    jge f_abs_done
    neg eax          ; Если отрицательное, делаем положительным
f_abs_done:
```

```
    ; Делим на 200
    cdq              ; Расширяем EAX до EDX:EAX
    mov ebx, 200
    idiv ebx         ; EAX = |x + y| / 200
```

```
f_end:
    pop ebx
    pop ebp
    ret 8            ; Очищаем стек от 8 байт
```

```
main:
    push ebp
    mov ebp, esp
```

```
    ; Обнуляем суммы
    mov dword [sum1], 0
    mov dword [sum2], 0
```

```
    xor esi, esi     ; ESI = индекс i
    xor ecx, ecx
    mov cx, [n]       ; ECX = n
    inc ecx           ; ECX = n + 1 (количество итераций)
```

```
loop_start:
    ; Первая сумма: накапливаем x[i]
    mov eax, [x + esi*4] ; EAX = x[i]
    add [sum1], eax      ; sum1 += x[i]
```

```
    ; Проверяем четность i
    test esi, 1        ; Проверяем младший бит
    jnz odd_case       ; Если i нечётное
```

```
even_case:
    ; i чётное: x_i / y_i + f(x_i, y_i)
    mov eax, [x + esi*4] ; EAX = x[i]
    xor edx, edx
    movzx edx, word [y + esi*2] ; EDX = y[i]
```

```
    ; Сохраняем для вызова функции
    push edx           ; Сохраняем y[i]
```

```
push eax          ; Сохраняем x[i]

; Деление x[i] / y[i]
cdq              ; Расширяем EAX до EDX:EAX
idiv dword [esp+4] ; Делим на y[i]
push eax         ; Сохраняем результат деления
```

```
; Вызов функции f(x[i], y[i])
; Аргументы уже в стеке (после push eax)
push dword [esp+8] ; y[i]
push dword [esp+8] ; x[i]
call f            ; Результат в EAX
```

```
pop ebx          ; EBX = результат деления (x[i]/y[i])
add esp, 8       ; Очищаем x[i] и y[i]
```

```
add eax, ebx     ; EAX = x[i]/y[i] + f(x[i], y[i])
add [sum2], eax  ; sum2 += результат
```

```
jmp loop_continue
```

odd_case:

```
; i нечётное:  $x_i + y_i^2 + f(x_i^2, y_i^2)$ 
mov eax, [x + esi*4] ; EAX = x[i]
add [sum2], eax      ; sum2 += x[i]
```

```
xor edx, edx
movzx edx, word [y + esi*2] ; EDX = y[i]
mov eax, edx          ; EAX = y[i]
imul eax, edx         ; EAX = y[i]^2
add [sum2], eax       ; sum2 += y[i]^2
push eax             ; Сохраняем y[i]^2
```

```
; Вычисляем x[i]^2
mov eax, [x + esi*4] ; EAX = x[i]
imul eax             ; EDX:EAX = x[i]^2
```

```
; Вызов f(x[i]^2, y[i]^2)
push dword [esp]     ; y[i]^2
push eax             ; x[i]^2
call f              ; Результат в EAX
```

```
add esp, 4          ; Убираем y[i]^2
add [sum2], eax     ; sum2 += f(x[i]^2, y[i]^2)
```

```

loop_continue:
    inc esi          ; i++
    dec ecx          ; ECX--
    jnz loop_start   ; Переход если ECX != 0

    ; r = sum1 + sum2
    mov eax, [sum1]
    add eax, [sum2]

    ; Вывод результата
    push eax
    push format
    call printf
    add esp, 8

    ; Выход из программы
    push 0
    call exit

```

Тестовые данные:

Точка 1	Точка 2	Количество целых точек между
1, 2	4, 5	2
0, 6	0, 3	2
0, 0	5, 7	0

Результат работы программы:

```

~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > nasm -f elf32 lab4_task3.asm -o lab4_task3.o
gcc -m32 lab4_task3.o -o lab4_task3 -no-pie
./lab4_task3
Количество целых точек между (1, 2) и (4, 5): 2
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > nasm -f elf32 lab4_task3.asm -o lab4_task3.o
gcc -m32 lab4_task3.o -o lab4_task3 -no-pie
./lab4_task3
Количество целых точек между (0, 6) и (0, 3): 2
~/projects/university/5sem/computer_architecture/lab4 on master >37 !6 ?9 > nasm -f elf32 lab4_task3.asm -o lab4_task3.o
gcc -m32 lab4_task3.o -o lab4_task3 -no-pie
./lab4_task3
Количество целых точек между (0, 0) и (5, 7): 0

```

Вывод:

В ходе выполнения лабораторной работы были изучены команды условного и безусловного перехода процессора x86, освоены методы организации циклов с помощью команд LOOP, LOOPE/LOOPNE, а также получены практические навыки создания подпрограмм с передачей аргументов через стек согласно соглашениям вызова. Реализованы три программы на ассемблере NASM: вычисление условного выражения с использованием команд условного перехода, вычисление суммы с вызовом функции и работой с массивами, а также решение задачи подсчёта целых точек на отрезке через алгоритм Евклида для нахождения НОД.